

Norman Dunbar

A Beginner's Guide to GitHub

A presentation to The T-Exchange, November
2026.

September 28, 2026

Me, Myself, Eye Publishing.

Contents

1	Introduction	1
1.1	What is Git	1
1.2	What is GitHub	2
2	Alternatives to GitHub	3
2.1	GitLab	3
2.2	CodeBerg	3
3	Very Simple GitHub	5
3.1	Repository URL Supplied	5
3.2	Searching	6
4	Simple Github	9
4.1	What to Do	9
5	Medium GitHub	11
5.1	Cloning a Repository	11
5.2	Updating the Cloned Repository	12
6	Slightly Advanced GitHub	13
6.1	Forking a Repository	13
6.2	Forking For Ourselves	14
6.3	Forking For Everybody	14
6.4	Keeping Our Fork Up-to-date	15
6.5	Raising a Merge Request	16
6.5.1	Raising the Merge Request	16
6.5.2	Actioned by the Original Authors	16
6.5.3	Checking the Merge has Completed	17
7	Advanced GitHub	19
	Appendices	21

A	Create a GitHub Account	23
B	Creating Repositories	25
C	HTTPS Credentials	27
D	SSH Credentials	29
D.1	Generate a New SSH Key	29
D.2	Add the New Key to the SSH Agent	30
D.3	Updating GitHub	30

Chapter 1

Introduction

In this, hopefully, short chat I will introduce you to GitHub. There will be a number of separate methods of using GitHub, as follows:

- *Very simple*: downloading a “release” with no interest in the source code.
- *Simple*: grabbing someone’s source code, as a Zip file, for our own use. This might not include a runnable binary—download a release as well if necessary.
- *Medium*: grabbing someone’s source code with the possibility of updating it, but *only* for our own use—we will “clone” a repository.
- *Slightly advanced*: grabbing someone’s code with the intention of updating it for ourselves and everyone else to use—we will “fork” a repository, then “clone” it.
- *Advanced*: here we look at creating websites for our *organisation*¹ and also for each individual project we have in GitHub.

1.1 What is Git

Git is a distributed version control system, created by Linus Torvalds the original creator of Linux.

The code for the Linux Kernel was held in a proprietary version control system but was given special permissions to use it for no, or little, charge.

Once fully embedded, the company decided to do the dirty on the Linux Kernel folks, so Linus went “stuff you” and wrote the original Git in a weekend.

Since then, Git has become probably the worlds best known and most used [distributed] version control system—there are a couple of others. *Mercury* for example.

A distributed version control system means that anyone using it for a project has a full copy of the source code *plus* the entire history of changes, releases and such like.

¹ By *organisation* I mean user or group as setup by ourselves, see later.

Work can be done off-line and away from the office, and uploaded later when back in contact, without suffering from *conflicts* if someone else edited the same files as you have.

Obviously, conflicts will arise if two or more folk edit the same *line(s)* of code. Git will help resolve the mess if it occurs.

1.2 What is GitHub

GitHub, <https://github.com>, is an online code repository, now owned by Microsoft, which offers individuals and/or organisations:

- Distributed version control of documents, code and almost anything else. Off-site backups in other words, plus the ability to undo horrendous cock-ups!
- Public and private repositories for projects.
- Web Pages for the organisation or individual.
- Web pages for individual projects.
- And much more: Wikis, Gists, Notifications, team working . . .

I won't be covering everything GitHub offers tonight, some of us will have beds to go to.

Other *Git* style code repositories are also available, the ones I'm familiar with being:

- GitLab, <https://gitlab.com>, is very similar to GitHub and not owned by Microsoft. My books live here. GitLab offers both private and public repositories.
- CodeBerg - A recent arrival for people who don't want Microsoft anywhere near their code especially with their so-called "AI" which literally steals anything and everything and claims it for its own!

CodeBerg, <https://codeberg.org>, only offers public repositories as it caters to free and Open Source projects only.

Chapter 2

Alternatives to GitHub

Some people and organisations are getting a bit huffy with GitHub since Microsoft took it over some time back, and especially since Microsoft and their so-called “AI” cannot be trusted not to steal your code and documentation and to claim it as their own.

2.1 GitLab

Gitlab is probably the oldest alternative to GitHub. It offers public and private repositories for your projects and is not (yet?) owned by Microsoft.

It is very similar to GitHub in what it offers. There is, as you might imagine a free and a pay-for-it option, the latter offering more support and is aimed at larger organisations.

Git lab is found at <https://gitlab.com>.

2.2 CodeBerg

CodeBerg is probably the newest kid on the block. It only caters to free and open source projects, is a non-profit and offers similar tools and features as GitHub.

CodeBerg is found at <https://codeberg.org>.

Chapter 3

Very Simple GitHub

The simplest use of GitHub is just to download some code, a document, a presentation or whatever. There's no need or wish to see the source code as you have no intention of looking at it, changing it or touching it in any way.

To access GitHub in this manner, you do not need an account, just the URL of the repository which holds the code that you are interested in, or hopefully, the username and project name.

For the sake of an example, we will consider using Norman Dunbar's "QL Assembly Language" eBook.

3.1 Repository URL Supplied

If you have been given the URL for the repository—<https://github.com/NormanDunbar/QLAssemblyLanguageMagazine>—you will need to open your browser there.

Obviously this shows the code, which is what you don't want, so look for "Releases" over on the right side of the screen. (There might be a number beside it, that's how many releases there are so far.)

- Click "Releases" or "Latest" and you will be taken to an area of GitHub where you can download whatever Norman Dunbar has released for this particular project.
 - "Latest" takes you directly to the most recent release of the QL Assembly Book.
 - "Releases" takes you to the list of all releases, in newest to oldest order.
- If there are no releases, you are up that famous creek—you might need to compile the code yourself, or find another utility to do the same task. (See Section 3.2, [Searching](#) below.)
- Where there are releases, beside the word "Assets", click the rightward pointing tiny arrowhead to open the list of assets provided in the release.

- As you do not want the source code, ignore the two assets with that name. Click on the PDF filename to download the eBook and also, if you want, click on the “code.zip” filename to grab all the code used in the book—if you want it that is.
- You can now go off, run yourself a nice hot bath, grab a glass of wine and settle back for a good read of the PDF file on writing assembly language for the 1980s Sinclair QL computer!

3.2 Searching

You don’t have the GitHub URL for the project, but you know it *might* have been someone named “Norman Dunbar” and you are pretty sure that the project had “Assembly” in it, somewhere. You need to search.

- Login to <https://github.com> in your browser.
- Press the forward-slash key (/) or click on the “Search” box near the top-right of the page.
- Type “Norman Dunbar”, with quotes and press return. The quotes forces a search for the text as given, not “Norman” or “Dunbar”. A page of results will hopefully be displayed. If all is well, you will find the user, otherwise. . .
- On the left side, under “Filter By”, click on “Users”. The result should change to show the desired user’ account.
- Click on the blue text of his name.

We have found the required user, but so far, not the code we want. We need to search Norman Dunbar’s repositories.

- On the NormanDunbar home page, press the forward-slash key again, or click on “Search”.
- You will see the search text box pre-loaded with “owner:NormanDunbar”. Add the word “Assembly” (without quotes).
- A list of possible matches is displayed One of which is `Assembly_Language_003.tex` and underneath that is the repository name—*QLAssemblyLanguage Magazine*, which looks like what we are after. Click it.
- A new page is displayed showing the title `QLAssemblyLanguageMagazine/Issue_003/Assembly_Language_003.tex` and the content of the file is displayed in the rest of the page.
- Click on the *QLAssemblyLanguageMagazine* word in the title to go to the main repository page.
- Now proceed as in Section 3.1, **Repository URL Supplied** above to download the required code.

If the search failed to find anything suitable, we have to scan through all of Norman Dunbar’s repositories.

- Make sure you are on the NormanDunbar home page as before. Click back through the previous pages if you have moved off of it.

- On the home page, click “Repositories” on the top left. The number adjacent to “Repositories” is the number you have to scan through to find the one you are hoping to find.
- Click “Sort” over to the right and select the “Name” option—the default is from most recently used to least recently used.
- Scroll through the list looking for something that looks like what you want. Your browser *might* let you search for text on the page, try CTRL-F and see what happens. You can search for the word “Assembly” which you think is in the repository name.
- Click “Next >” at the bottom if you didn’t find it. Repeat as necessary until you either do find it, or run out of repositories!

Chapter 4

Simple Github

Simple GitHub usage is when we look around the code repository, or click on a link, and we then simply download the code for our own use. The salient points are:

- We don't need (or want) an account on GitHub. We certainly do not wish to login.
- We don't intend to amend the code, just use it, although we might want to read through it for our own edukashun¹.
- If we did change the code in any way, we would not be able to upload it to GitHub to the original authors.

4.1 What to Do

We will either have been supplied with a link to the code repository on GitHub, or, we will know what to search for.

- Connect to <https://github.com> or to the URL supplied—for example, <https://github.com/NormanDunbar/QLAssemblyLanguageMagazine>.
- Click the green “<> Code” button.
- Click the “Download ZIP” option.
- Wait . . .
- Unzip and use the code as desired—bear in mind that you might have to compile the code if the download does not contain a compiled binary. Those normally live in “Releases” but can sometimes also be found in the main repo itself.
- If the binary you wanted is not in the repo, check for any releases as discussed in Chapter 3, [Very Simple GitHub](#), and download the binary from the releases area.

¹ It's like an education, but better! ;o)

Chapter 5

Medium GitHub

This manner of using GitHub is quite common, and we won't need an account to be able to proceed.

- We wish to download code for our own use.
- We may intend to make some changes to suit our needs, but only for ourselves.
- We don't intend to upload our changes to the original authors as we are a particular niche case!

We can either:

- Download the Zip file with the source code, and get a release if necessary, as already discussed in Chapter 3, [Very Simple GitHub](#) and Chapter 4, [Simple Github](#).
- Alternatively, "clone" the repository to our PC.

5.1 Cloning a Repository

Assuming the procedures outlined in Chapter 3, [Very Simple GitHub](#) or Chapter 4, [Simple Github](#) are not enough for us, then proceed as follows to clone the repository and its full change history, onto our PC.

- Locate the required repository using any supplied URL or by searching for it.
- Click on the "<> Code" button as before.
- Click "HTTPS"
- Click the icon to copy the repository URL.
- Open a terminal session on your PC.
- Navigate to wherever you keep your source code, for me it's all kept under `SourceCode`.
- Within the `SourceCode` directory, type `git clone` and paste in the URL just copied, to get something like `git clone https://github.com/NormanDunbar/QLAssemblyLanguageMagazine.git`. Press return.

- The GitHub repository will be cloned to your PC, creating a new sub-directory named `QLAssemblyLanguageMagazine` inside your `SourceCode` directory.
- Now, go play with the code. Make changes, recompile it, have fun!

Remember, you have no ability to upload your changes to the original repository.

5.2 Updating the Cloned Repository

You can, from time to time, update your clone of the repository quite simply.

- Open a terminal session on your PC.
- Navigate to the `SourceCode/QLAssemblyLanguageMagazine` sub-directory.
- Type the command “git pull” and press return.
- Any updates made by the original author(s) will be downloaded to your clone, *overwriting what was there before*—it’s best not to do this if you have made any important changes!

Chapter 6

Slightly Advanced GitHub

In this chapter, we want to get hold of a copy of a repository and make changes to it. After testing—very important—we want to upload our changes.

We have two choices here.

- Create a “fork” of a repository for our own use to prevent changes made to the original from updating ours unless we choose to apply them.
- Create a fork, but use it to offer our changes back to the original.

Either way, we will need a GitHub account before we can proceed. The rest of this document assumes you have one.

6.1 Forking a Repository

The process to fork a repository—take a full copy and work on it ourselves, outwith the control of the original authors—is actually quite simple.

- Login to GitHub and find your way to the home page for the repository you want to fork.
- On the same line as the repository title, towards the right side, click on “fork”.
- On the next page—Create a new fork—check that the name defaulted is suitable, change it if not.
- Click “Create Fork” button.
- Wait . . .
- A new page appears, showing the home page for *your* fork of the original project.

Now that we have a fork, we can proceed to work on it. We have two choices here:

- Work on the code for our own benefit, but keep changes to ourselves.
- Work on the code, perhaps fixing bugs or making improvements, for our own and everyone’s benefit.

6.2 Forking For Ourselves

If we want to make changes and keep them to ourselves, and we do not want any updates to the original repository to overwrite our changes, we need to:

- Fork the required repository as per Section 6.1, [Forking a Repository](#).
- Clone the fork onto our local PC as per Section 5.1, [Cloning a Repository](#)
- Work on the clone:
 - Create branches for changes. `git checkout -B my_branch`
 - Edit the code, compile & test. Repeat!
 - Schedule the changes:
 - Add everything. `git add -all`
 - Alternatively, add named files. `git add <list of files>`
 - Commit said changes. `git commit -m "What I did here"`
- Merge commit(s) into the main branch.
 - Checkout “main” branch. `git checkout main`
 - Merge the branch changes. `git merge my_branch`
- Push commits back to our fork. `git push`
- Delete the now unwanted branch. `git branch -D my_branch`
- Optionally, “tag” the new state of the repository, perhaps for a release? `git tag -a -m "Release 1.00" Rel_1.00`—Don’t have spaces in the tag name! Alternatively, `git tag Rel_1.00`.

That’s all we have to do. Any changes we make will never be overwritten by changes made to the original that we might subsequently apply to our fork. There is a slight chance of a conflict if we change the same lines as the original authors, but that’s fairly simple to resolve.

Maybe, one day, we decide we want to share our changes with the original authors. How do we do that?

6.3 Forking For Everybody

Making our changes available back to the original authors for *possible* inclusion in the original repository is very similar to the processes outlined in Section 6.2, [Forking For Ourselves](#):

- Fork the required repository as per Section 6.1, [Forking a Repository](#).
- Clone the fork onto our local PC as per Section 5.1, [Cloning a Repository](#)
- Work on the clone:
 - Create branches for changes. `git checkout -B my_branch`
 - Edit the code, compile & test. Repeat!

- Schedule the changes:
 - Add everything. `git add -all`
 - Alternatively, add named files. `git add <list of files>`
- Commit said changes. `git commit -m "What I did here"`
- Merge commit(s) into the main branch.
 - Checkout “main” branch. `git checkout main`
 - Merge the branch changes. `git merge my_branch`
- Push commits back to our fork. `git push`
- Raise a “merge request” back to the original repository. See Section 6.5, **Raising a Merge Request** below.
- Wait for the merge to be approved or rejected.
- Only now, can you delete the now merged branch. Don’t forget to make sure that you are not currently checked out on the branch to be deleted. `git checkout main` followed by `git branch -D my_branch`
- Optionally, “tag” the new state of the repository, perhaps for a release? `git tag -a -m "Release 1.00" Rel_1.00`—Don’t have spaces in the tag name! Alternatively, `git tag Rel_1.00`.
- If you decide to tag the repository, as it is now, but are not making a release, you can still tag it. `git tag -a -m "tagging my changes" my_changes`. `git tag -list` will display all known tags.

6.4 Keeping Our Fork Up-to-date

Keeping our fork up to date with the changes made to the original repository is surprisingly easy.

- Login to our own github account.
- Open the repository’s home page.
- By the title there will be a notice that “*this repository is nn commits ahead of, and nn commits behind <original repository>*”.
- Click the “Sync Fork” button.
- That’s it!

Now that our repository has been updated, we need to update the local clone on our PC.

- Start a terminal session.
- Navigate to the location where you created the clone.
- Execute the command `git pull`.
- That’s all!

6.5 Raising a Merge Request

After our changes have been made, tested, merged and pushed to our fork, we are now ahead of the original repository. If we don't care about passing changes back to the original, this is fine, we can delete our temporary branch as it is no longer needed.

However, if we do wish to share our changes with the original authors, we need to raise a *Merge Request*. The processes are described next.

6.5.1 Raising the Merge Request

To raise a merge request, aimed back at the original repository, proceed as follows:

- Login to your account and open the fork's home page within your account.
- Click "Pull requests" near the top left.
- Click "New pull request"—it's a green button.
- Check or fill in the following:
 - Base repository: Username/repository name for original repository.
 - Base: Master or Main as appropriate. It's a drop down, so select it.
 - Head repository: Your user/Fork name
 - Compare: Pick your branch for this change.
- When done, look for "Able to merge". indicating all is well. Otherwise, there are fixes to be made!
- Click "Create pull request"—it's a green button again.
- Make sure the title and descriptions are meaningful and complete with all information appropriate.
- Click "Create pull request" below the description.

At this point, it is out of your control. *DO NOT* delete the branch you created—the original repository needs it to merge your changes. Just leave it alone.

You might have to wait some time. Be patient.

6.5.2 Actioned by the Original Authors

The original development team responsible for the repository will get a notification that a pull request has been raised. They will, in an ideal world:

- Click on "Pull requests".
- Clicks on your pull request.
- Sees a message that there are no "No conflicts", which means they can merge your changes.
- Reads, checks the changes, your code standards etc.

- S/he/they might request further information. S/he/they might need some specific tests, disclaimers etc.
- Eventually, clicks “Merge pull request”.
- Updates the title and description.
- Clicks “Confirm merge”.

That’s it. Your changes are merged, everybody in the world has access to them and can sing your praises for evermore! (Perhaps.)

6.5.3 Checking the Merge has Completed

At some point, you will get a notification when you login. Click the “in-tray” icon, top right to see your notifications.

One of them will be regarding the merge request. Click it. You will see whatever the original authors write in the description of the merge request when they applied it.

In addition, you will no longer see your merge request in the “pull requests” area of your fork. You can check them though:

- Click “Pull requests”
- Change the search criteria from “is:pe state:open” to “is:pr state:closed” or “is:pr state:merged”
- Check the request(s) that appear to read the details.

Once you have confirmed that your change has been merged and the pull request has been closed, you can delete the branch, finally.

Chapter 7

Advanced GitHub

GitHub offers you free web hosting for organisations/individuals and for individual projects through something called “GitHub Pages”.

There is a test version of the T-Exchange’s (old) website—<https://moraymakerspace.com>—based on WordPress which currently takes up at least half a Gigabyte. The test website is currently 115 Mb in size and has extra posts! You can play with it at <https://the-t-exchange.github.io>.

Note

The test website is not 100% identical. It is missing the contact form for now, Twitter/X images have been converted so that they display as images—you can’t click through to Twitter at the moment. The usual “T-Exchange” header image needs to be inserted into the code that builds the web pages. (I’m working on that).

However, it is smaller, faster and free! It is also possible to redirect our URL, <https://moraymakerspace.com> to point at the GitHub one <https://the-t-exchange.github.io>.

GitHub also offers free Wikis for projects and organisations/individuals, there are also “gists” which are small notes, code snippets and such like.

These are most likely going to be the subject of a further talk, if enough interest can be raised.

Appendices

Appendix A

Create a GitHub Account

Creating a GitHub account is quite simple. However, if you are creating a secondary account, you must be logged out of your primary account in the browser, otherwise, things will get confusing! To sign out:

- Click your profile picture in the top right corner.
- Click “Sign Out”
- Confirm by clicking one or other of the subsequent “sign out” buttons.

Ok, the slate is clean, lets create an account.

- Point your browser at <https://github.com>.
- Click “Sign up”.
- On the right:
- Fill in your email address.
- Fill in an initial password, be secure! You need either 15 characters, or, 8 characters which must include at least one digit and at least one lowercase letter.
- Fill in a username. Replace spaces with hyphens—but only one consecutive hyphen is allowed. You will be told if the username is available or not.
- Select a country.
- Tick the box for email updates if you want them. You don’t have to though. Your choice.
- Click “Create account”.

You are now prompted for a nine digit code. Check the email address that you supplied and enter the digits. Once you have done so, you will need to login with the username (or email) and password you supplied above.

Job done, you can now create repositories—see Chapter B, [Creating Repositories](#)—and the like. Read on, however, for details of how to use *https* and *ssh* protocols to be able to push changes back to a repository.

Appendix B

Creating Repositories

Creating a repository is simple.

- Login to your account on GitHub.
- On the top menu-bar, towards the right side, Click the + (plus) button.
- Click “New repository”.

On the following screen, fill in details specific to the new repository.

- Give the repository a name. You will be told if you have used it previously.
- Give a description too. Be meaningful. The description here will be written into a `README.md` file if you choose to have one.
- Make the repository visible as required.
- Add a README if required. The default is not to have one, but they are very handy, and, will be used to display details of the repository on its home page.
- The `.gitignore` file is a good one to have as it stops files you might not want to be uploaded for public visibility, from being uploaded. Depending on how your project will be generated or compiled, you can choose a suitable `.gitignore` from the drop-down list.
- Add a licence¹—there are many to choose from.
- Click “Create repository”.

That’s it. All you need to do now is:

- Add and edit files, online, in the GitHub page for the repository.
- It is much easier, for anything more than quick changes, to clone the repository to your own PC and work on it there, pushing committed changes back as and when appropriate. See Section 5.1, **Cloning a Repository** for details.

¹ Or License if you spell things the American way!

Appendix C

HTTPS Credentials

When you clone a repository to your PC using the HTTPS protocol (the default when you click the “Download Code” button) you will be able to clone the repository to your PC without any problems.

When you make and commit changes, attempting to `git push` them back to your repository will ask for your username and password. The push will fail with a message telling you:

```
remote: Invalid username or token.
Password authentication is not supported for Git operations.
fatal: Authentication failed for
'https://github.com/<username>/<repository name>.git/'
```

The solution is to either use SSH, See Chapter D, [SSH Credentials](#) or create a Personal Access Token or PAT, as follows:

- Login to your account on GitHub.
- Click your profile picture on the far right.
- Click “Settings”.
- On the left sidebar, click “Credentials”.
- Back to the right, click “Personal Access Token (Classic)”.
- Click “Generate New Token”
- Click “Generate New Token (Classic)”
- Add a note of what it is for. Be meaningful!
- Set an expiry date, as appropriate.
- Tick “repo” & “Workflow” at the very least. You will also want to add some of the other options as appropriate, *read them carefully!* I suggest “gist”, “Notifications” and *maybe* “delete_repo” (oooh! Brave!)
- Scroll down and click “Generate token”.

Warning

The next screen to appear will list the generate PAT for your use. Make a copy of it *right now* because you will never see it again!

Back on your laptop or PC, you can try the `git push` exercise again. This time, when prompted for a password, you must supply the PAT you generated instead.

However, before doing that, do this:

```
git config [--global] credential.helper store
```

If you leave “--global” out of the command, the credential will be stored for the current repository clone only. Otherwise, it will apply to any and all repositories from the username you generated it under.

Now when you type the password (and where you use the PAT as the password) it will be stored and you won’t need to remember it in future.

Warning

The PAT will be stored on your computer in *plain text* so consider the security implications if you decide to store the PAT.

Appendix D

SSH Credentials

The preferred method of accessing GitHub remotely, is to use SSH. On Mac and Linux systems, it's already installed but the *mega-secure* Windows operating system needs to have an SSH installed. OpenSSH is recommended.

D.1 Generate a New SSH Key

Windows users will have a screen to fill in, and I can't tell you what is asked for as I don't have Windows these days. You need to generate a new "ED25519" key linked to your email address.

On Mac and Linux:

```
ssh-keygen -t ed25519 -C 'your_email@whatever.com'
```

You will be asked to save the key pair in a file. The default is `/home/norman/.ssh/id_25519` on my system. For the sake of this example, I will use `/home/norman/.ssh/github_norman_id_25519` as I keep track of what the key is used for in the filename.

When prompted for a passphrase, just press return. And again when prompted to confirm the pass phrase.

Warning! Warning! Warning!

The key pair generated consists of a private key and a public key. The names are a clue and the only one of the two that should ever be given out, uploaded or whatever, is the *public* one. The file that ends with `.pub` is the public one of the two. **Never, ever, ever allow the private key to be seen, extracted, uploaded or given away in any way, shape or form.**

D.2 Add the New Key to the SSH Agent

We need to start up the `ssh-agent`, if it's not already running, and add our keyfile to it. *Do not* use the filename with `.pub` on the end.

```
eval `ssh-agent -s`  
ssh-add ~/.ssh/github_norman_id_25519
```

That's our local PC configured. Next, we need to update our Github account.

D.3 Updating GitHub

We first must copy the content of the *public* key of the key pair. This happens on our PC of course.

```
cat ~/.ssh/github_norman_id_25519.pub
```

Highlight and copy the entire text that is displayed on screen. It will resemble the following—which is not any of my own keys in case you were wondering!

```
ssh-ed25519 YADaYadaYaDa . . . uggawugga norman@example.com
```

- Login to your account on GitHub.
- Click your profile picture on the far right.
- Click “Settings”.
- On the left sidebar, click “SSH and GPG Keys”.
- Back to the right, click “New SSH key”.
- Add a title, be meaningful.
- Paste the text of your public key into the “Key” text-box.
- Click “Add SSH key”

Now, and in the future, when you click on the green “Download code” button to get a repository's URL, click first on “ssh” and then copy the URL. The SSH URL's resemble:

```
git@github.com:username/repository.git
```

You will be able to push code changes back to github without specifying a username and/or password—your public and private keys will do all the necessary negotiations.